Unit 4

Policy based methods train the policy directly w/o an intermediate value function

↳ Parameterize the policy - using a nn. π_θ, the policy will output a probability distribution over all actions.

$$\pi_\theta(s) = P[A \mid s; \theta]$$

The policy given a state outputs a prob. distribution over actions at that state.

- maximize performance of the policy using gradient ascent by controlling the parameter θ.
- Define an objective function $J(\theta)$ → expected cumulative reward. Want to find θ that maximizes this function.
- Policy-based methods - on-policy most of the time optimized by maximizing the local approximation of the objective function.
- Policy-gradient methods (a subclass of policy-based methods) optimize θ directly by performing gradient ascent on the performance of $J(\theta)$.

→ Advantages of policy-gradient methods

- Simplicity of integration - estimate the policy directly w/o storing additional data (action values).
- can learn a stochastic policy while value functions can't.
  ↳ don't need to implement exploration vs. exploitation.
  ↳ no more perceptual aliasing (when 2 states seem the same but need dif. actions).

...

↳ more effective in higher-dimensional action spaces & continuous action spaces.

↳ have better convergence properties — if the Q value of one action rises above another that drastically changes behavior. policy updates are more gradual.

Disadvantages of policy-gradient methods.

↳ frequently converge at a local maximum rather than a global optimum.

↳ training is slower,

↳ have a higher variance.

Parameterized stochastic policy - outputs a distribution over A based on θ. Probability of taking each action is called the action preference.

goal is to have good actions be sampled more often.

Training Loop:

Collect an episode w/ π

Calculate the return (sum of rewards)

Update the weights of π:

- if positive return → ↑ the prob. of eq. (state, action) pairs taken during the ep.
- if negative return → ↓ the probs.

To know if the policy is good, it is evaluated using the score/objective function $J(\theta)$

The **objective function** gives the performance of the agent given a **trajectory** (state action seq. w/o considering rew. (unlike an episode)), and it outputs the expected cummulative reward.

$$J(\theta) = E_{\tau \sim \pi}[R(\tau)]$$

$$R(\tau) = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} \cdots$$

$R(\tau)$: cummulative reward; τ: trajectory

The **expected return** (expected cummulative rew.) = weighted average, where weights are given by $P(\tau;\theta)$ of all possible values that $R(\tau)$ can take

$$J(\theta) = \sum_{\tau} P(\tau;\theta) R(\tau)$$

$R(\tau)$: cummulative return from traj.

Calculate return by summing for all trajectories the prob. of taking that trajectory given θ and the return of that trajectory

$P(\tau;\theta)$: Probability of the traj. (depends on θ and it defines the policy that it uses to select the actions of the traj.

$$J(\theta) = \sum_{\tau} P(\tau;\theta) R(\tau)$$

$$P(\tau;\theta) = \left[\prod_{t=0} P(s_{t+1}|s_t, a_t)\, \pi_\theta(a_t|s_t)\right]$$

$P(s_{t+1}|s_t, a_t)$: Environment Dynamics (state distribution)

$\pi_\theta(a_t|s_t)$: probability of taking action a_t at state s_t

Maximize the objective function

$$\max_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$$

goal is to find θ that maximizes the expected return.

Update step for gradient ascent

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

↳ can't calculate the true gradient (computationally too expensive) — uses a sample-based estimate

$P(\tau;\theta)$ may not be differentiable b/c it may be an unknown

Policy Gradient Theorem

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau)]$$

The Reinforce Algorithm (Monte Carlo Reinforce)

a policy gradient algorithm that uses an estimated return from an entire episode to update θ.

use π_θ to collect τ

use to estimate gradient $\hat{g} = \nabla_\theta J(\theta)$

$$\nabla_\theta J(\theta) \approx \hat{g} = \sum_{t=0} \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau)$$

- $\approx$: estimation of the gradient for a single traj.
- $\log \pi_\theta(a_t|s_t)$: probability to select a_t given s_t.
- $R(\tau)$: cummulative return
- $\nabla_\theta \log \pi_\theta(a_t|s_t)$: direction of the steepest increase of the log prob.